

Abstract geometric lines in black on a white background, forming various overlapping polygons and shapes.

Secure Coding Review – Java Login
System
CodeAlpha Cyber Security Internship
Task 3: Secure Coding Review
Reviewed Application: Java Console
Login System
Prepared By: Urooj Fatima Memon

OBJECTIVE

To identify security vulnerabilities in a Java login application and provide recommendations for secure coding practices.

APPLICATION OVERVIEW

APPLICATION DESCRIPTION

A simple Java-based login system that:

- Accepts username and password from the user
- Verifies credentials
- Grants or denies access

Technologies Used

- Java
- IntelliJ IDEA
- Console-Based Application

Login Process

- User enters username
- User enters password
- System validates credentials
- Login success or failure message is displayed



VULNERABILITIES FOUND!!

1. HARDCODED CREDENTIALS:

```
IF (USERNAME.EQUALS("ADMIN") &&  
PASSWORD.EQUALS("ADMIN123"))  
ISSUE: CREDENTIALS ARE STORED  
DIRECTLY IN SOURCE CODE.
```

2. PLAIN TEXT PASSWORDS:

PASSWORDS ARE STORED AND
COMPARED AS PLAIN TEXT.
ISSUE: PASSWORDS CAN BE EXPOSED
IF SOURCE CODE IS ACCESSED.

3. NO LOGIN ATTEMPT LIMIT:

UNLIMITED LOGIN ATTEMPTS ARE
ALLOWED.
ISSUE: ENABLES BRUTE-FORCE
ATTACKS.

4. WEAK PASSWORD POLICY
NO PASSWORD COMPLEXITY REQUIREMENTS.
ISSUE: USERS MAY CHOOSE WEAK PASSWORDS.

Security Risks

1.Credential Theft:

Attackers can view hardcoded usernames and passwords.

2.Unauthorized Access:

Weak authentication increases the risk of account compromise.

3.Brute Force Attacks:

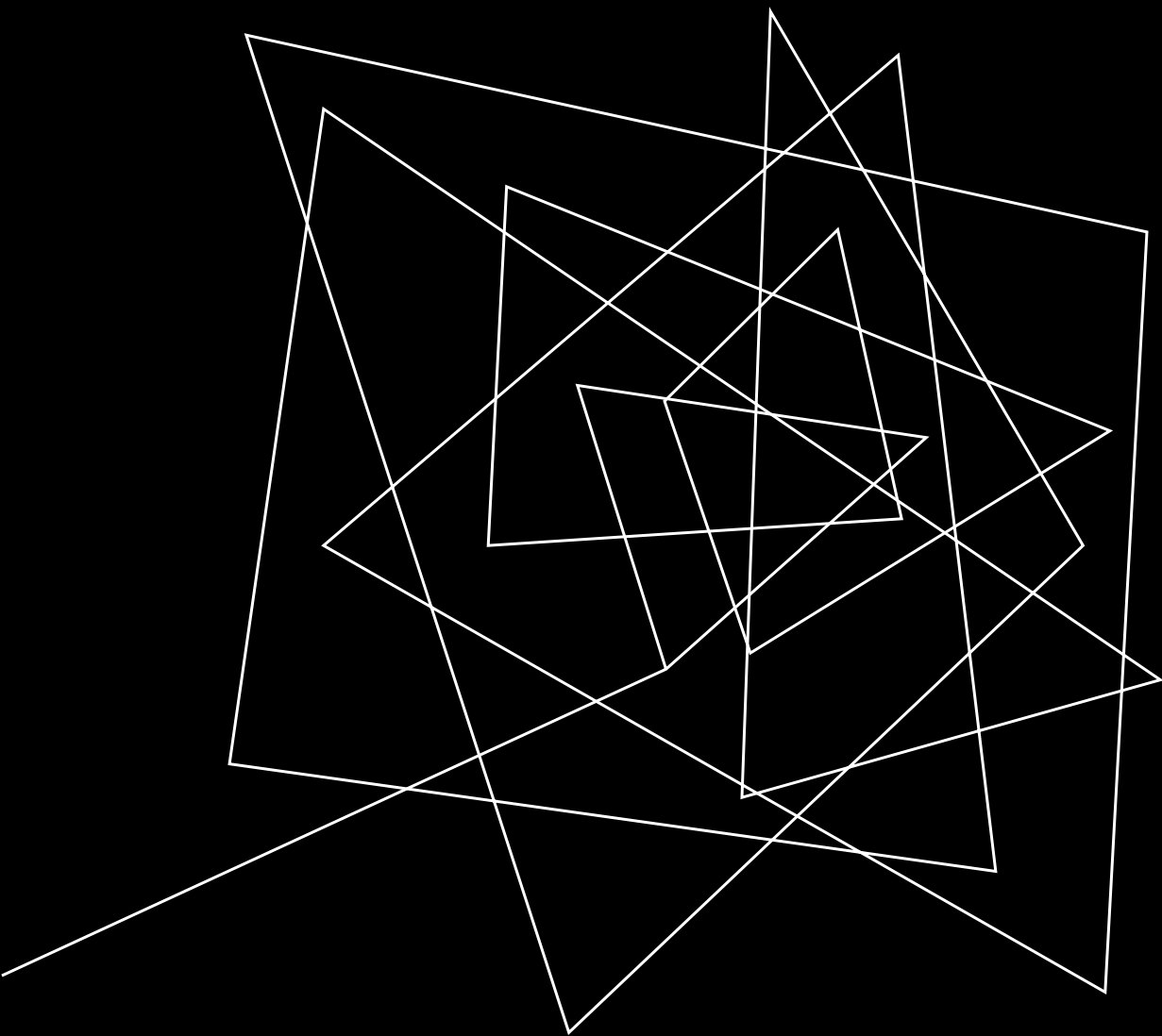
Unlimited login attempts allow attackers to guess passwords.

4.Data Breaches:

Compromised accounts may expose sensitive information.

5.Reputation Damage:

Security vulnerabilities can reduce user trust



Recommendations

Use Password Hashing:

Store hashed passwords using SHA-256 or bcrypt.

Implement Login Attempt Limits:

Example:

- Maximum 3 failed attempts
- Temporary account lockout

Enforce Strong Passwords:

Require:

- Uppercase letters
- Lowercase letters
- Numbers
- Special characters

• Secure Credential Storage:

• Store credentials in a database instead of source code.

• Validate User Input:

• Check and sanitize all user inputs before processing.



SECURE VERSION & CONCLUSION

Improvements Implemented

- ✓ Password hashing using SHA-256
- ✓ Better credential protection
- ✓ Improved authentication security
- ✓ Secure coding best practices

CONCLUSION

The review identified several security weaknesses in the original Java login system, including hardcoded credentials, plaintext passwords, and the absence of login restrictions. Implementing secure coding practices such as password hashing, input validation, and account protection mechanisms significantly improves application security and reduces the risk of cyberattacks.

A series of white, thin, overlapping geometric lines on a black background, forming a complex, abstract pattern on the left side of the slide.

THANK YOU!
ANY QUESTIONSS?